



ScholarLog

M5 – Composants de Traitement et Visionnage Scans

Auteur :
Hofer Lukas
Technicien ES en informatique 2^e année

Responsables du module :
Choffat Lise
Claude Fahrni
Montavon Dominique

Table des matières

1. Contexte et Objectifs.....	3
2. Problématique : Pourquoi ne pas stocker et lire le PDF nativement ?.....	3
3. Architecture de la Solution (Pipeline de Traitement).....	4
3.1 Phase d'Ingestion (Enregistrement du document).....	4
3.2 Phase de Restitution (PdfViewerControl).....	4
3.3 Phase d'Exportation (Reconstruction du document).....	4
4. Justifications Techniques Détaillées.....	5
4.1 WebP dans CBZ.....	5
4.2 Séparation asynchrone (Task.Run)	5
4.3 Stratégie et Choix du Format d'Exportation.....	5
5. Limitations et Compromis assumés.....	6
6. Tests.....	6
7. Conclusion	7

1. Contexte et Objectifs

Dans le cadre du projet ScholarLog, l'application doit permettre aux étudiants de gérer leurs évaluations et de conserver des traces de leurs documents localement. L'application repose sur une architecture de type Monolithique utilisant une base de données locale SQLite. L'objectif de ce composant est de permettre l'enregistrement, l'affichage et l'exportation de documents PDF liés aux évaluations, tout en respectant deux contraintes majeures :

- Empreinte mémoire (RAM & Stockage) minimale : La taille de la base de données SQLite ne doit pas exploser (objectif estimé à ~60 Mo maximum pour l'ensemble des scans).
- Interface non-bloquante : L'interface développée en Avalonia UI doit rester 100% fluide, même lors du traitement de documents lourds.

2. Problématique : Pourquoi ne pas stocker et lire le PDF nativement ?

L'approche classique consisterait à enregistrer le fichier .pdf brut dans un champ BLOB de la base de données SQLite et à utiliser une librairie de visionnage PDF standard. Cette approche a été rejetée pour les raisons suivantes :

- Poids des données : Un PDF scanné classique pèse rapidement plusieurs mégaoctets. Stocker ces fichiers bruts dans SQLite provoquerait une fragmentation rapide et ralentirait considérablement les requêtes de base de données.
- Lourdeur des librairies Avalonia : Avalonia UI étant multi-plateforme, les bibliothèques capables de rendre nativement du PDF (rasterization vectorielle) sont souvent très lourdes, ou incompatibles avec Linux/macOS sans dépendances complexes.
- Gel de l'interface (UI Thread Blocking) : Le décodage d'un PDF lourd en mémoire sature rapidement le CPU, ce qui provoque des saccades ("freezes") dans l'interface graphique si cela n'est pas géré avec une ségrégation stricte des threads.

3. Architecture de la Solution (Pipeline de Traitement)

3.1 Phase d'Ingestion (Enregistrement du document)

Lorsqu'un utilisateur charge un PDF, le pipeline suivant est déclenché de manière 100% asynchrone :

- **Extraction** : Le fichier PDF est lu en flux continu (Stream) et chaque page est extraite sous forme d'image rasterisée (SKBitmap via SkiaSharp).
- **Downscaling (Redimensionnement)** : L'image est redimensionnée dynamiquement pour ne pas dépasser un seuil fixe (2.5 Mégapixels).
- **Compression Agressive** : L'image est encodée au format WebP avec une qualité basse/moyenne (qualité fixée à 45). Ce format offre un ratio qualité/poids largement supérieur au JPEG pour les documents scannés.
- **Archivage RAM** : Ces images WebP sont regroupées dans une archive ZIP générée à la volée en mémoire (format CBZ).
- **Persistance** : C'est ce tableau d'octets (byte[]) ultra-compressé qui est sauvegardé dans la base SQLite via Entity Framework Core.

3.2 Phase de Restitution (PdfViewerControl)

Pour l'affichage, l'interface graphique utilise un composant Avalonia personnalisé (PdfViewerControl.axaml).

- Le contrôle reçoit le byte[] depuis la base de données.
- Il ne décompresse pas toute l'archive d'un coup. Il stocke uniquement les octets WebP compressés de chaque page dans une liste légère en RAM (List<byte[]>).
- Lorsqu'une page est affichée, une seule image est décodée à la fois.

3.3 Phase d'Exportation (Reconstruction du document)

Puisque le document n'est pas stocké sous sa forme PDF originale, l'exportation nécessite une recomposition à la volée.

- **Extraction RAM** : Le service extrait les flux d'images WebP depuis le BLOB en mémoire.
- **Dessin Vectoriel** : Le module de traitement recrée un document PDF vierge (SKDocument) et y dessine les images redimensionnées page par page.
- **Sauvegarde OS** : Le fichier final est enregistré sur le disque de l'utilisateur via le StorageProvider d'Avalonia, garantissant la compatibilité des boîtes de dialogue avec tous les systèmes d'exploitation (Windows, macOS, Linux).

4. Justifications Techniques Détaillées

4.1 WebP dans CBZ

Pourquoi le format WebP dans un conteneur CBZ ?

Efficacité spatiale : Le WebP compressé à 45% transforme une page PDF de 2 Mo en une image de quelques dizaines de kilo-octets. Le conteneur Zip (CBZ) ajoute une compression supplémentaire sur les métadonnées. L'empreinte en base de données devient négligeable.

Sécurité et Portabilité : Le stockage d'un byte[] compressé au lieu de chemins d'accès garantit que l'étudiant ne perdra pas ses documents s'il déplace son fichier SQLite.

4.2 Séparation asynchrone (Task.Run)

Dans Avalonia, toute modification de l'interface doit se faire sur le Thread principal (Dispatcher.UIThread). Cependant, l'encodage WebP est une tâche dite "CPU-Bound".

- Si le décodage se fait sur le thread UI, l'application fige.
- La solution : Les opérations de compression et de conversion sont encapsulées dans des Task.Run() couplés à des IAsyncEnumerable. L'interface ne reçoit l'instruction de mise à jour visuelle (via Dispatcher.UIThread.Post()) que lorsque l'image est totalement prête dans les threads de travail en arrière-plan.

4.3 Stratégie et Choix du Format d'Exportation

Il est crucial de distinguer le format de stockage interne du format d'exportation externe. Bien que l'archive CBZ soit techniquement optimale pour la base de données, l'application exporte systématiquement les documents au format PDF. Ce choix repose sur l'interopérabilité :

- Standard scolaire : Le PDF est le format de prédilection demandé par les institutions pour la remise de devoirs ou la transmission de justificatifs.
- Accessibilité : Peu d'utilisateurs possèdent un lecteur d'archives CBZ (format principalement utilisé pour les bandes dessinées). En revanche, le PDF est lu nativement par tous les navigateurs web. Ainsi, la complexité technique de la conversion (PDF -> CBZ -> PDF) est totalement masquée à l'utilisateur, qui conserve l'illusion de manipuler des fichiers PDF standards de bout en bout.

5. Limitations et Compromis assumés

Cette conception radicalement orientée vers l'optimisation du stockage implique certains compromis techniques :

1. Perte du texte natif : Le PDF original étant rasterisé en images, il n'est plus possible de sélectionner le texte à la souris dans l'application.
2. Altération des données à l'export : Le fichier récupéré ne sera pas le PDF source original, mais une version scannée et compressée de celui-ci.

Ce compromis est jugé acceptable vis-à-vis du cahier des charges, car la fonction première est la conservation d'une trace d'évaluation (comme un document papier scanné), justifiant ainsi la priorité absolue donnée à la légèreté de la base de données SQLite embarquée.

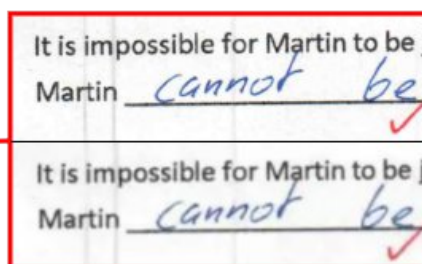
6. Tests

Voici les différents tests :

- Un scan d'un livre de 396 pages
- 3 épreuves moyenne

6.1 Résultats

Objet	Original (PDF)	Restitution (PDF)	Stockage dans BDD (BLOB)	Gain dans la DB
Livre	95,5 Mo	50,0 Mo	17,28 Mo	82 %
Épreuve de 8 Pages	2,80 Mo	1,20 Mo	0,518 Mo	82 %
Épreuve de 7 Pages	2,00 Mo	0,20 Mo	0,319 Mo	84 %
Épreuve de 8 Pages	2,50 Mo	0,76 Mo	0,364 Mo	85 %



Original

Restitution

7. Conclusion

En conclusion, ce système de gestion des documents remplit parfaitement ses deux objectifs principaux : prendre le moins de place possible sur l'ordinateur, tout en gardant une application rapide et agréable à utiliser.

En compressant intelligemment les documents en arrière-plan plutôt que de les sauvegarder de manière classique, l'application parvient à réduire l'espace de stockage nécessaire de plus de 80 %.

L'emprunte mémoire estimé de la BDD SQLite avec des documents de 8 pages :

$$1_{\text{travail de diplôme}} * 0.5 Mo_{\text{taille en BDD}} + 7_{\text{modules}} * (3_{\text{leçons}} * 3_{\text{épreuves}} + 1_{\text{Rapport de module}}) * 0.5 Mo_{\text{taille en BDD}} = 35.5 Mo$$

Pour l'étudiant, toute cette mécanique reste totalement invisible. Lorsqu'il souhaite récupérer ou envoyer un travail, le système lui restitue simplement un fichier PDF standard, lisible sur n'importe quel appareil.